School of Computer Science, McGill University

COMP-512 Distributed Systems

Assignment 1*, Due on 16-October

Exercise 0: Group Work / NO AI

1. Meeting times *Please indicate the dates and times (from/to) when you discussed the solutions of this assignment together.*

2. No AI declaration: *Confirm that you did not use AI by writing the following sentence, replacing X with the names of the two group members: "We, X, did not use any form of AI to solve this assignment".*

Exercise 1: P2P algorithm (10 Points)

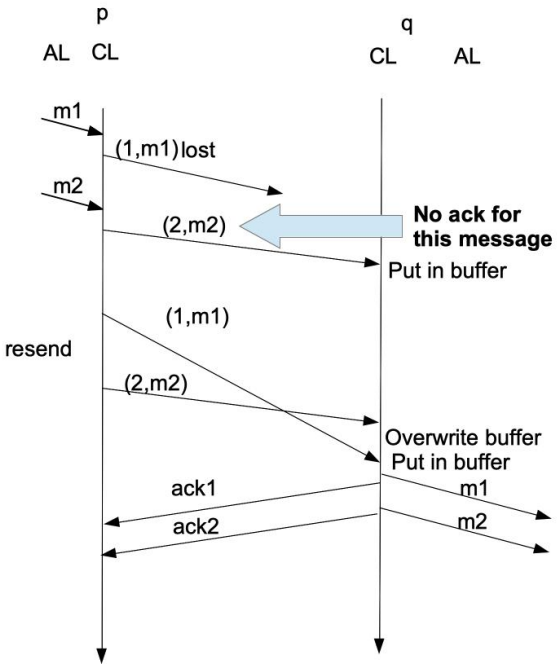
The quiz contained the following algorithm:

Algorithm 1 Algorithm Outline

Process p (Sender)	**Process q (Receiver)**
1: **Upon start communication with** q	1: **Upon start communication with** p
2: senderList := $\emptyset$	2: buffer := $\emptyset$
3: ctr := 0	3: nextToDeliver := 1
4: **Upon Send(m) request from appl. layer**	4: **Upon receiving** (id, m) **from** CL of p
5: ctr := ctr + 1	5: **if** $id \geq$ nextToDeliver **then**
6: add (ctr, m) to senderList	6: buffer[id] := m
7: n_send((ctr, m)) to CL of q	7: **while** buffer[nextToDeliver] contains a message **do**
	8: deliver buffer[nextToDeliver] to appl. layer
8: **Upon receiving** $ack(id)$ **from** CL of q	9: n_send($ack(nextToDeliver)$) to CL of p
9: remove (id, m) from senderList	10: nextToDeliver := nextToDeliver + 1
	11: **end while**
10: **Every timeout interval**	12: **else**
11: **for all** $(id, m) \in$ senderList **do**	13: n_send($ack(id)$) to CL of p
12: n_send((id, m)) to CL of q	14: **end if**
13: **end for**	

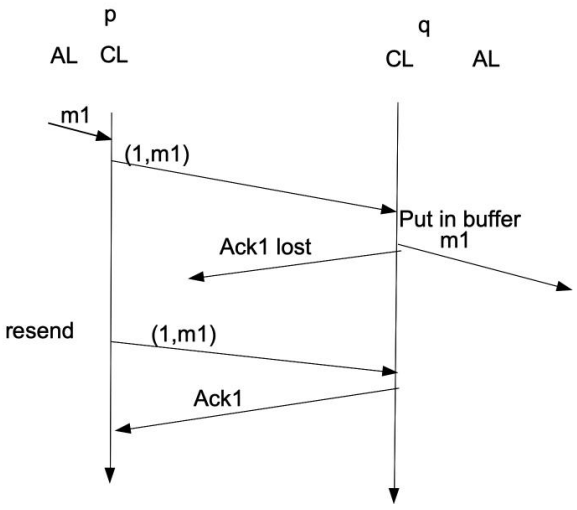
1. *Given above algorithm, show an example execution where a message (id, m) that the CL of p submits to the underlying unreliable point-to-point (i.e., (id, m) is transmitted as a "physical" message through the network to q) but the CL at q does not send an acknowledgement for this particular physical message.*

*Copyright: Bettina Kemme. This information is for personal use only and it is strictly forbidden to share it with anybody else outside the students of this class or post it to any platform.



Solution Outline:

2. *Given above algorithm, show an example execution where CL at q sends multiple ack(id) for a specific value of id (that is, multiple acks are sent for one logical message m that the application at p had submitted).*



Solution Outline:

Note that you can draw the executions by hand and include a photo of your drawing in the solution pdf.

Exercise 2: Time (25 Points)

1. Assume a computer with a software clock ct that represents time as an 8-byte long integer and that should maintain monotonicity. Assume now the clock is updated using the following simple mechanism: every time interval (triggered after a certain time has passed on the hardware clock of the computer), ct is increased by 1 (i.e., a simple $ct := ct + 1$).

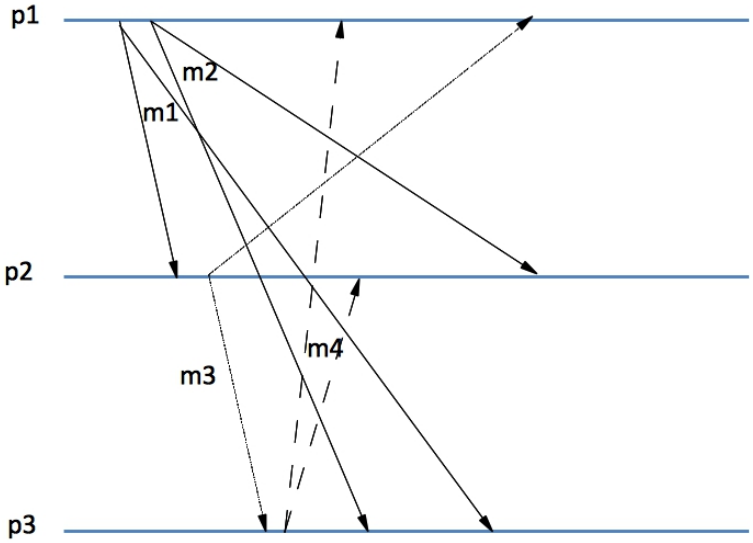
   Assume now that at the last update, ct was set to 1,007,655,996.

   Now assume the computer receives notification from an external time server, and realizes its clock should actually be 1,007,655,980 (after adjusting the time-value it receives from the time server to take into account message delay).

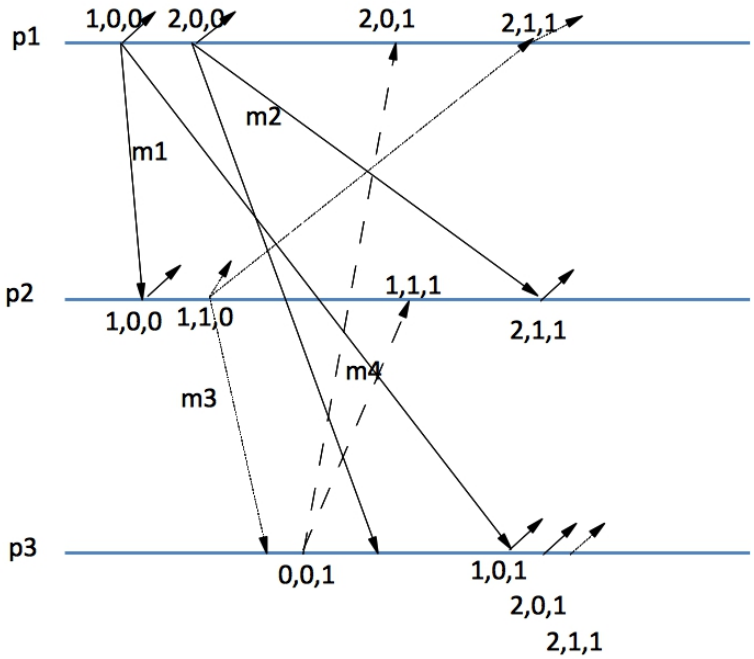
   What is the best value to set ct next time the time interval expires? Provide a short reasoning.

   Solution Outline: *1,007,655,996 (keep it at the value it was); That's the only correct way to move forward with integer increases. Just simply only increase by one every two updates.*

2. Below figure shows a system with three processes. Assume each horizontal line represents the time at the group communication layer of the process. It accepts messages submitted from the application (not shown) and sends them to the group communication layers of the other processes. Each group communication layer delivers the message to the application when causal order can be obtained. Note that this is exactly the example we discussed in class.



   (a) The group communication system implements causal order using vector clocks as described in lecture notes 512-gc on page 19. At the beginning, the vector clock at each process is $(0, 0, 0)$.
   Show for each message and each process, when the group communication layer delivers the message to the application (similar as in the slides and as we have discussed in class with an arrow up from the group communication layer). Also, show at each process each vector clock change.

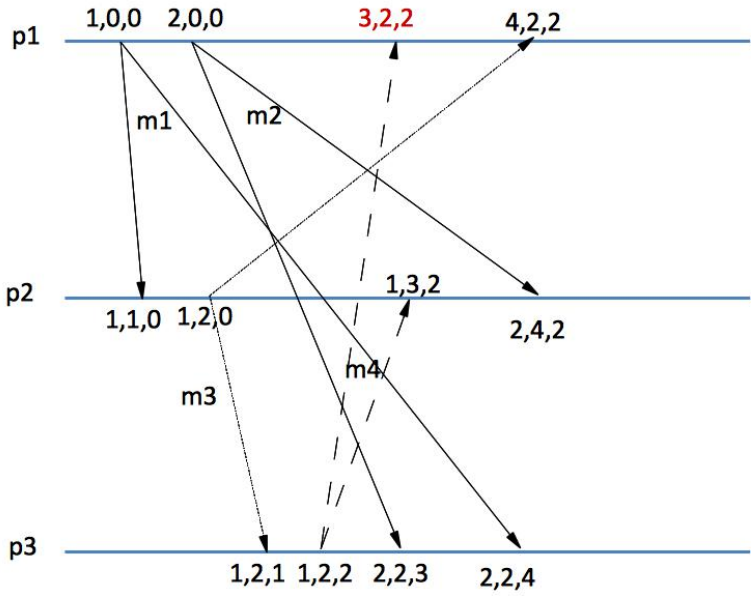


Solution Outline:

(b) *For each set of messages, indicate whether they are concurrent or there is a causal order between them.*

Solution Outline: $m1 \rightarrow m2$, $m1 \rightarrow m3$, $m1||m4$, $m2||m3$, $m2||m4$, $m3||m4$

(c) Now just assume that we want to timestamp events in class as we discussed in the lecture notes 512-time about time. Taking again above figure, we have three processes p1, p2 and p3 that exchange messages. There are only two types of events at each process. The *send* of a message to the other two processes, and a *receive* of a message from another process. There are no local or other events. *Timestamp each of the events according to the vector clock algorithm described on slide 19.*



Solution Outline:

Exercise 3: Causal Order Multicast (50 points)

In the causal order multicast described in class each message piggybacks the entire local vector clock. If a process p_i CO-multicasts a message m piggybacking V_i, $V_i[i]$ is p_i's sequence number for m (since $V_i[i]$ is increased exactly upon every multicast operation). $V_i[j], j \neq i$, represents p_j's sequence number of the last message CO-multicast by p_j, and delivered by p_i before CO-multicasting m.

1. Now assume p_i CO-multicasts a message m_1 piggybacking V_i such that $V_i[j] = x$ for some $j \neq i$. After that, it CO-multicasts m_2 with again $V_i[j] = x$. That is, between CO-multicasting m_1 and m_2, p_i has not delivered any message CO-multicast by p_j. As such, it might not be necessary to piggyback the $V_i[j]$ component again. A process p_k receiving m_2 will only deliver m_2 after it has delivered m_1. And it will only deliver m_1 once it has delivered the message m CO-multicast by p_j with sequence number x. Hence, it has automatically delivered m before m_2.
   That is, instead of piggybacking the entire vector V_i, p_i piggybacks on message m a sequence of tuples $(p_j, V_i[j])$, (p_j is process identifier). We only include a tuple for a process $p_j, j \neq i$ if p_i has delivered a message CO-multicast by p_j since the last time p_i CO-multicast a message. Additionally, $V_i[i]$ is included as p_i's sequence number of the message. [1]

   Develop an algorithm that uses above format to piggyback the information relevant to determine the causal order. Assume there is an underlying simple multicast primitive that provides reliable delivery (i.e., no message is lost) but messages might arrive in any order.

   Hint: start with the algorithm discussed in class and adjust to your needs. Make sure to keep track when messages are delivered at p_i after p_i has CO-multicast its last message in order to know which tuples to piggyback. Furthermore, make sure to deliver all messages only when causally preceding messages have been delivered.

   Solution Outline: ***Algorithm at process*** p_i*:*

   1. *Data structures (For process i):*
      - *int* $V_i[N]$*: vector clock of* p_i
      - *Tuple* $(p_j, V_i[j])$*:* $V_i[j]$ *is the local time at* p_i *when* p_i *received the last message from* p_j*.*
      - $AckSet_i$*: set of tuples to be acknowledged. Contains only tuples created since last message was multicast*
   2. *On initialization (For process i):*
      - $V_i[j] := 0$ *(j=1,2, ..., N)*
      - $AckSet_i := \emptyset$
   3. *Upon CO-multicast(g,m) from application layer (For process i):*
      - $V_i[i] := V_i[i]+1$*; // increase local clock entry*
      - *multicast* $(g, < V_i[i], AckSet_i, m >)$
      - *CO-deliver(g,m) immediately to its own application layer*
      - $AckSet_i := \emptyset$
   4. *Upon receiving* $(g, < ts, AckSet_j, m >)$ *from* p_j
      - *if(j==i) then ignore; endif*
      - *wait until* $(ts == V_i[j]+1)$ *and* $\forall < p_k, t > \in AckSet_j : t \leq V_i[k]$

[1] Although we do not need to piggyback all vector clock entries, we now have to add the process identifiers. Assuming that a process identifier needs as much space as a vector clock entry, we will only then save space if we haven't delivered any message from at least half of the members since the last CO-multicast. This will be true for busy senders that usually receive messages from only few sites between consecutive CO-multicasts.

- *CO-deliver (g,m) to the application*
- *$V_i[j]$=$V_i[j]$+1;*
- *if $\exists X$, such that $(p_j, X) \in AckSet_i$, remove (p_j, X) from $AckSet_i$*
- *add $< p_j$, $V_i[j] >$ to $AckSet_i$ // p_i has received all messages from p_j up to $V_i[j]$*

2. You can even further reduce the information needed to be piggybacked. Let's have a look at above algorithm in the following case. Assume p_i CO-multicasts message m_1. After that, p_i receives and delivers a message m from p_j and sets $V_i[j] = x$. After that, p_i receives and delivers a message m' from p_k and sets $V_i[k] = y$. Assume m' causally follows m. Now p_i CO-multicasts m_2. In above algorithm, m_2 will piggyback (p_j, x) and (p_k, y) indicating that these two messages (and, of course, m_1) have to be delivered before m_2. However, it is actually only necessary to piggyback (p_k, y). Since m' causally follows m, any other site will deliver m' only after m. Hence, it is enough to make sure that m_2 will be delivered after m' to make sure that it will also be delivered after m. Let's have a more concrete example with the following messages.
m_1: (sender: p_1, seq: 1)
m_2: (sender: p_2, seq: 1)
m_3: (sender: p_2, seq: 2, ack: $(p_1, 1)$)
m_4: (sender: p_3, seq: 1, ack: $(p_2, 2)$)
m_5: (sender: p_4, seq: 1, ack: $(p_1, 1), (p_2, 1)$)
In above example everybody has to deliver m_2 before m_3 (messages from the same sender are delivered in increasing id order). m_3 indicates additionally that p_2 has delivered m_1 before CO-multicasting m_3. Hence, everybody has to deliver m_1 before m_3. There is no ordering requirement of m_2 in regard to m_1 since they are concurrent. m_4 indicates that p_3 has delivered m_3 before CO-multicasting m_4. Hence, everybody has to deliver m_3 before m_4, and with it, automatically m_1 before m_4. As such it is not necessary for m_4 to piggyback information about m_1. m_5, CO-multicast after m_1 and m_2 were delivered, has to indicate both these messages since they are not causally related to each other.

Develop an algorithm that uses above format to piggyback the information relevant to determine the causal order. Assume there is an underlying simple multicast primitive that provides reliable delivery (i.e., no message is lost) but messages might arrive in any order.

Note: In my solution, the second algorithm is a simple extension of the first algorithm. But whether this is possible, of course, depends on your first algorithm.
Solution Outline: ***Algorithm at process*** *p_i:*

1. *Data structures (For process i):*
   - *int $V_i[N]$: vector clock of p_i*
   - *Tuple (p_j, $V_i[j]$): $V_i[j]$ is the local time at p_i when p_i received the last message from p_j.*
   - *$AckSet_i$: set of tuples to be acknowledged. Contains only tuples created since last message was multicast*
2. *On initialization (For process i):*
   - *$V_i[j]$: = 0 (j=1,2, ..., N)*
   - *$AckSet_i := \emptyset$*
3. *Upon CO-multicast(g,m) from application layer (For process i):*
   - *$V_i[i]$:=$V_i[i]$+1; // increase local clock entry*
   - *multicast (g, $< V_i[i], AckSet_i, m >$)*
   - *CO-deliver(g,m) immediately to its own application layer*
   - *$AckSet_i := \emptyset$*

4. *Upon receiving (g, $< ts, AckSet_j, m >$) from p_j*
   - *if(j==i) then ignore; endif*
   - *wait until (ts==$V_i[j]$+1) and $\forall < p_k, t > \in AckSet_j : t \leq V_i[k]$*
   - *CO-deliver (g,m) to the application*
   - *$V_i[j]$=$V_i[j]$+1;*
   - *$\forall < p_k, t > \in AckSet_j$: if $\exists < P_k, t' > \in AckSet_i \wedge t == t'$, then remove $< p_k, t' >$ from $AckSet_i$.*
   - *if $\exists X$, such that $(p_j, X) \in AckSet_i$, remove (p_j, X) from $AckSet_i$*
   - *add $< p_j, V_i[j] >$ to $AckSet_i$ // p_i has received all messages from p_j up to $V_i[j]$*

3. *For each of the algorithms that you developed, is there a scenario where the size of the additional information piggybacked with an application message is larger than if we had used the original causal order multicast? If yes, describe this scenario. If not, explain why this cannot happen.*

Exercise 4: Trade-offs (15 points)

We discussed four total order multicasts in class. One is not on the lecture slides. There, one process, let's say p_1 is again a "sequencing process". It keeps a counter. When p_i, $i \neq 1$ wants to multicast a message, it sends a *requestSequenceNumber* to the sequencer process p_1. p_1 increases its counter and sends the value back to p_i. p_i then adds this value to the message and sends it to everybody. Everybody delivers messages in the order of their sequence numbers. When p_1 wants to multicast a message, no request message is needed, it can simply increase the counter locally and use the value as sequence number for the message, and send the message to everybody.

1. *For this sequencer algorithm that is depicted in the class notes (not the one above) and assuming N processes, indicate*

   (a) The number of messages hops needed before delivery. That is how many messages will be sent one after the other from the time the AL submits the multicast request and a process delivers the message to the application. Consider the scenarios (a) the sequencer process p_1 is the sender and we consider delivery at process p_i, $i \neq 1$; (b) a process p_i, $i \neq 1$ is sending and message delivery at p_1, (c) p_i, $i \neq 1$ is sending and message delivery at p_j, $j \neq$.

   (b) The number of messages sent across the network for one application message when the sender is p_i, $i \neq 1$.

   Solution Outline: *1(a) 1 (message and sequence message can be sent concurrently before the first is received; (b) 1; (c) 2; 2: 2*(N-1)*

2. *For each of the 4 algorithms, indicate one aspect that you see as an advantage compared to at least two other algorithms. Consider things like number of hops, number of messages sent per application message, bandwidth needed, potential bottleneck, complexity etc.*

   Solution Outline: *Examples: Token: no bottleneck compared to all others, Sequencer described above: simple to implement compared to token and sequencer from lecture nodes, Coordinator based: very simple and only 2-hop delay (compared to token and sequencer above), Sequencer from lecture slides: 2-hop delay compared to token and sequencer from above.*